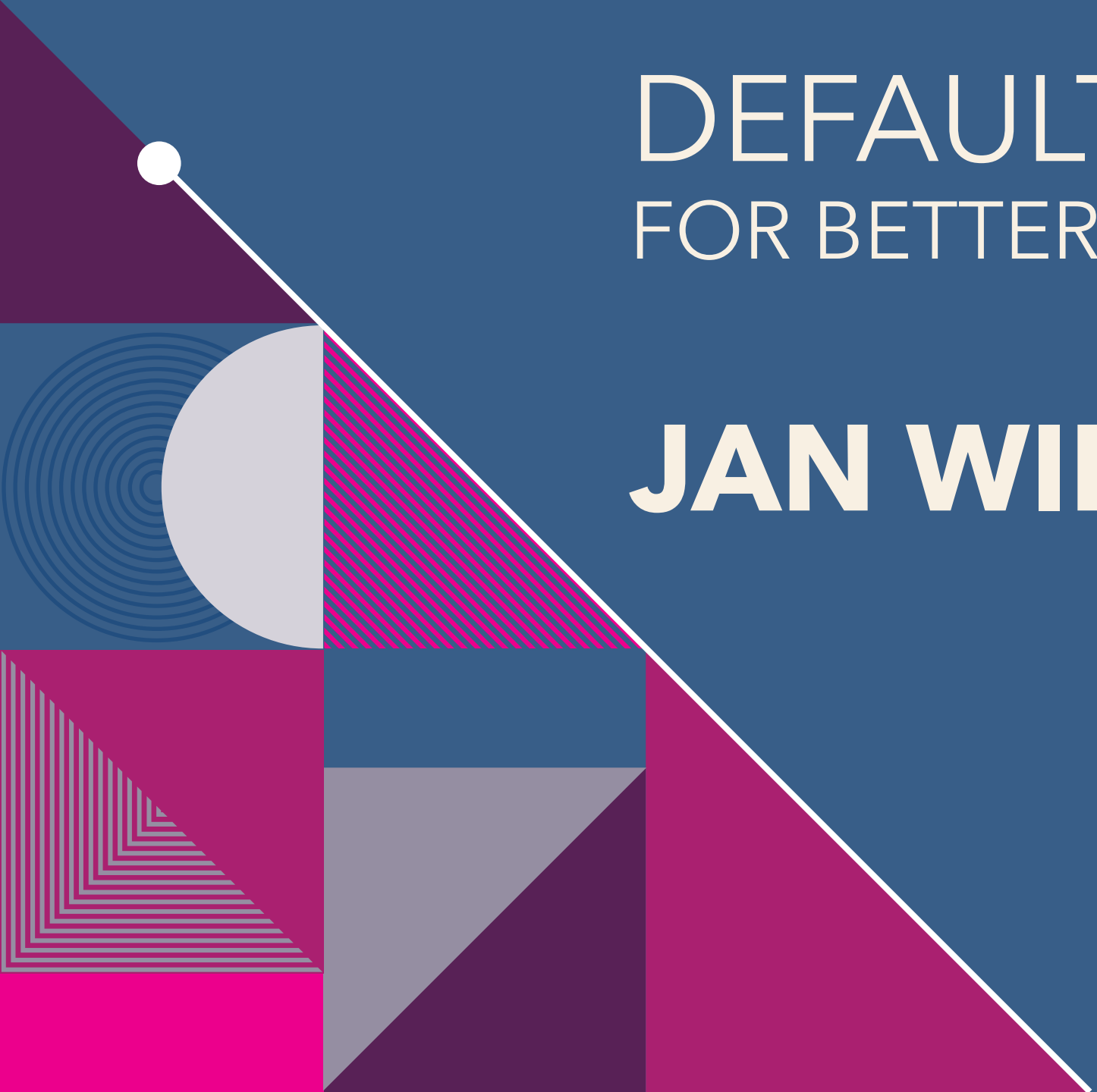
DEFAULT GUIDELINES
FOR BETTER C++

JAN WILMANS



C++ UNDER THE SEA 2025



About me

- Software developer for 20+ years
- Contributor to WG21, SG14 and SG1
- Vimec: Quality Inspection of Medical Glass
- C++Coach / Software Architect for Vimec
- Co-Organizer C++ Meetup Eindhoven

https://www.meetup.com/nl-NL/cpp_meetup_eindhoven/



https://nullptr.nl
https://vimec.nl





2

THE GOAL: DEFAULT GUIDELINES

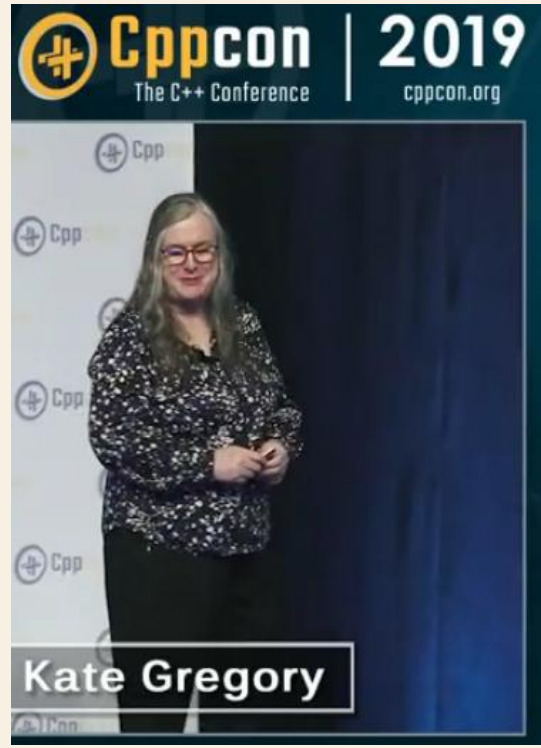
Use standard C++ and make it **expressive, consistent, and opinionated** – promoting **clarity** and **safety** through conventions rather than new syntax.

Not all guidelines are coding guidelines

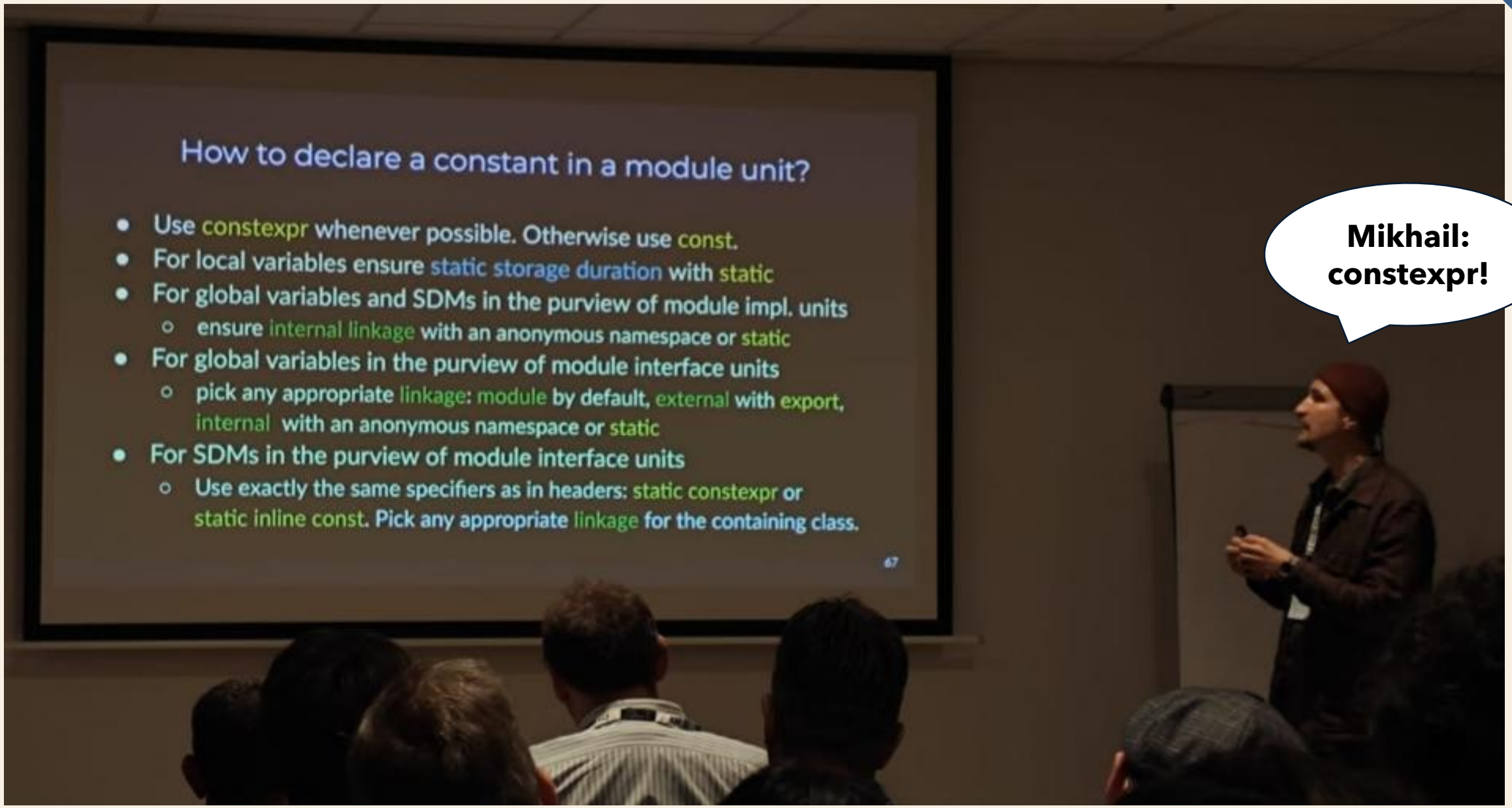
- Some are style recommendations
- Some are part of the development process
- Some are c++ code related

UNDERSTANDABLE CODE

- **Make it more reader-friendly**
  - Use verbs in **function names** (express action)
  - Use nouns in **class and variable names**
  - Maintain **consistent formatting (**use clang-format / astyle)
  - Minimize **implicit conversions** (warnings can help)
- **Make Local reasoning possible**
  - Limit variable scope
  - Minimize **shared state** (and avoid global state)
  - Prefer functions that do one thing (single responsibility)
  - Prefer **const** correctness!



[CppCon 2019: Kate Gregory "Naming is Hard: Let's Do Better"]

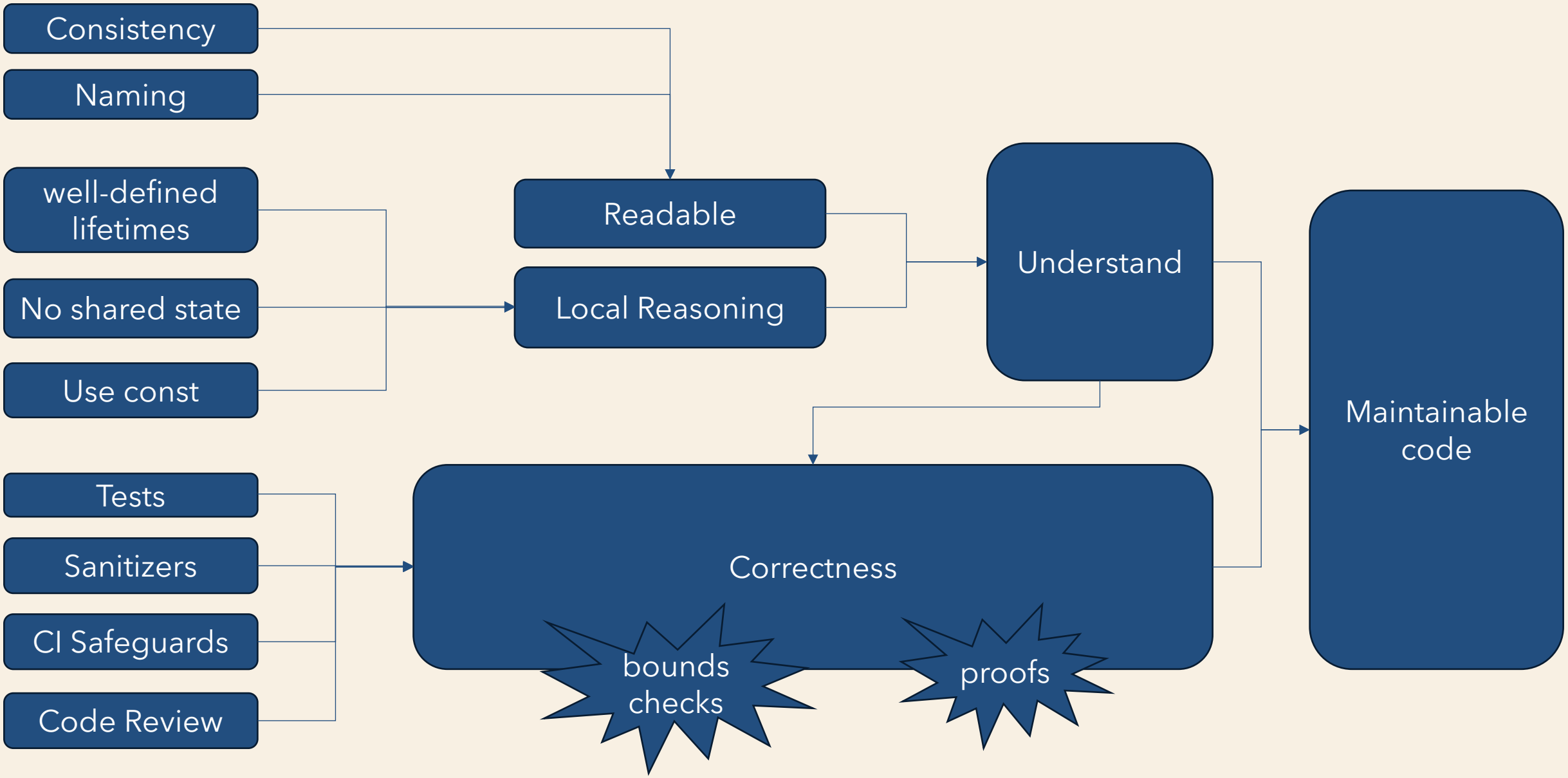

How to declare a constant in a module unit?
Use constexpr whenever possible. Otherwise use const.
For local variables ensure static storage duration with static
For global variables and SDMs in the purview of module impl. units
ensure internal linkage with an anonymous namespace or static
For global variables in the purview of module interface units
pick any appropriate linkage: module by default, external with export, internal with an anonymous namespace or static
For SDMs in the purview of module interface units
Use exactly the same specifiers as in headers: static constexpr or static inline const. Pick any appropriate linkage for the containing class.
67
Mikhail: constexpr!

MAINTAINABLE CODE

Relies on correctness, which is a hard problem, but can be supported by process and tools:

- Code reviews
- Tests
- Build time checks
  - Compiler warnings
  - Sanitizers
  - Spellings checkers (for comments)
  - **Safeguards** in Continuous Integration (CI)

An incomplete diagram



WHAT IS STOPPING US?

- **Just** right good™ code? **Wait**, what does good code look like?
- The answer may vary....
- C++ serves many domains and developers, so it avoids prescribing a single way of doing things.
- It favors flexibility and sometimes even **safety**! over enforcing a programming style, mostly for the sake of **performance**.
- C++ views backwards compatibility an important **asset**.
  But it has also become known for being the language that has:

the wrong defaults™

COMMON HEADACHES

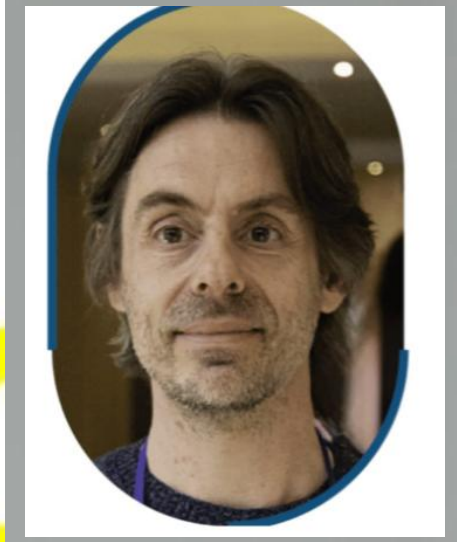
- manual memory (resource) management
- globally defines types and variables (un-scoped enums)
- non-virtual destructors
- unsafe access by default (unchecked boundaries)
- implicit conversions
- interestingly: in C++ "**default initialization**" means "**no initialization**"
- lack of **const** member functions
- mutable variables (arguments, locals and members) that never change
- discarded return values (no error checking)
- unscoped standard library classes, like thread and mutex (fixed by **std::jthread** and std::lock_guard)

Most have to do with Lifetime, Scope and Lack of local Reasoning.

C++20 Thread Improvements



Lieven de Cock



WHERE TO FIND GOOD DEFAULTS?

- Company guidelines (often domain specific)
- C++ Core Guidelines (1104? and counting)
- Experience

QUOTE FROM THE C++ CORE GUIDELINES:

Rules with no enforcement are unmanageable for large code bases. Enforcement of all rules is possible only for a small weak set of rules or for a specific user community.

- ~~But we want lots of rules~~, and we want rules that everybody can use.
- ~~But different people have different needs.~~
- But people don't like to read lots of rules.
- But people can't remember many rules.

yes

So, we need subsetting to meet a variety of needs.

- But arbitrary subsetting leads to chaos.

yes

In.force: Enforcement

LET'S CREATE A SMALL SET OF WEAK RULES!

- I did not invent new guidelines

- They were **gathered** from existing material
  - **filtered** for clarity
  - **simplified** to be practical
  - **prioritized** to set defaults

- Simplification makes them more accessible
- Prioritization gives ready-to-use **defaults**

C++ DEFAULT GUIDELINES

- These **defaults** are an ordered subset of common Guidelines that I use on daily basis.
- You **do not** necessary need use **this set** of **defaults**, the important thing is to **have** a set of defaults
- Guidelines are not hard Rules, use them responsibly!

GUIDELINES, NOT RULES

- If you can use a default, use it.

- If the default does not seem fitting, that is great, because that means that you already are thinking about the problem!

When you are not following the defaults, it might be a great time to:

- introduce a **function** whose name explains **what** is happening.
- add a **comment** explaining **why** you decided to deviate.

1. TURN ON WARNINGS! AND USE SANITIZERS

A good start ....

GCC

```
-Werror -Wconversion -Wsign-conversion -Wall -Wextra -Wpedantic -Wdeprecated
-fsanitize=address,undefined
```

Clang

```
-Werror -Wconversion -Wsign-conversion -Wall -Wextra -Wpedantic -Wdeprecated
-fsanitize=address,undefined
```

MSVC

Just having **/W3** is a really good start, but **/W4** is even better.

```
/W4
```

Notice: we do not have **`-Wconversion -Wsign-conversion`** as errors, but for new project they are great.

MOAR WARNINGS

GCC and CLANG

```
add_compile_options(
  -Wall
  -Wextra
  -pedantic
  -Wcast-align
  -Wformat=2
  -Wmisleading-indentation
  -Wmissing-include-dirs
  -Wnull-dereference
  -Wswitch-enum
  -Wunused
  -Wunused-variable
  -Wunused-parameter
  -Wvla
  # Warnings that are not allowed in C compilation units
  $<$<COMPILE_LANGUAGE:CXX>:-Wnon-virtual-dtor>
  $<$<COMPILE_LANGUAGE:CXX>:-Woverloaded-virtual>
)
```



MOAR WARNINGS

MSVC

```
/W4
/w14242 /w14254 /w14263 /w14265 /w14287 /w14289 /w14296 /w14311 /w14545 /w14546
/w14547 /w14549 /w14555 /w14640 /w14826 /w14905 /w14906 /w14928
```



DEALING WITH EXISTING WARNINGS

Example:

We have a bunch of conversion of ‘int’ to ‘size_t’ warnings in existing code.
To make sure we do not introduce warnings in other categories, we use **-Werror**, but allow **sign-conversion** to be a **warning** for the time being.

-Werror

-Wno-error=sign-conversion

-Wno-error=<warning> turns that specific rule back into a warning even if -Werror was specified

RUN TESTS WITH SANITIZERS

```
6227  summary for 1156 known issues:
6228   1028 opencanary
6229  summary for issues with count == 0:
6230      0 cast-align
6231  new kinds of issues that were introduced:
6232      maybe-uninitialized
6233  tr_regression ends with exit code 1
6234  $ if [ -n "${CREATE_PACKAGE}" ] ; then ./translation/scripts/collect_language_tables > language.log ; fi
6235  $ if [ -n "${CREATE_PACKAGE}" ] ; then ./installation/scripts/generate_cpack_packages.sh ${BUILD_OUTPUT_DIR} ${PACKAGE_DIR} ; fi
6236  $ verification/gather_size_information.py ${BUILD_OUTPUT_DIR} metrics.json
6237  Total: 8075.03 MB in 324 file(s)
6238  Average: 24.92 MB
6239  Running after_script
6240  Running after script...
6241  $ echo "===" ; cat summary.log || true; echo "==="
6242  ===
6243  Sanitizer issues:
6244  Sanitizer summary:
6245  ASAN: 0 issues (0 unique)
6246  UBSAN: 0 issues (0 unique)
6247  TSAN: 0 issues (0 unique)
6248  ===
```

COLLECT REPORTS FROM CI BUILDS

```
452 Summary of all issues:
453 deprecated-literal-operator: 1                  : []: [LOW] identifier '_sha256_hash_value' preceded by whitespace in a literal operator declaration is
454 float-overflow-conversion: 1                    : []: [LOW] implicit conversion from 'float' to 'bool' changes value from 3.4028235E+38 to true
455 float-zero-conversion: 1                        : []: [LOW] implicit conversion from 'float' to 'bool' changes non-zero value from -3.4028235E+38 to true
456 shadow-field: 1                                 : []: [LOW] parameter 'position' shadows member inherited from type 'ModuleBase'
457 unused-template: 1                              : []: [LOW] unused function template 'ToDoubleValues'
458 declaration-after-statement: 2                  : []: [LOW] mixing declarations and code is incompatible with standards before C99
459 pre-c23-compat: 2                              : []: [LOW] 'bool' is incompatible with C standards before C23
460 range-loop-bind-reference: 2                   : []: [LOW] loop variable '[index, item]' binds to a temporary value produced by a range of type
461 reserved-macro-identifier: 2                   : []: [LOW] macro name is a reserved identifier
462 suggest-destructor-override: 2                 : []: [LOW] '~VMeasurementRow' overrides a destructor but is not marked 'override'
463 unused-member-function: 2                      : []: [LOW] unused member function 'DefineImage'
464 cast-qual: 3                                   : []: [LOW] cast from 'const unsigned char *' to 'char *' drops const qualifier
465 implicit-fallthrough: 3                        : []: [LOW] unannotated fall-through between switch labels
466 reserved-identifier: 3                         : [IPC]: [LOW] identifier '_simpl_get_hostname' is reserved because it starts with '_' at global scope
467 unreachable-code: 3                           : []: [LOW] code will never be executed
468 disabled-macro-expansion: 4                   : []: [LOW] disabled expansion of recursive macro
469 inconsistent-missing-destructor-override: 5   : []: '~GevEx' overrides a destructor but is not marked 'override'
470 AP#11: 6                                      : [device]: [LOW] Anti-pattern: do not use extern
471 AP#17: 6                                      : []: [LOW] Using namespace found in header file
472 AP#4: 6                                       : []: [LOW] Anti-pattern: do not use volatile
```

```
520 sign-conversion: 7164                          : []: implicit conversion changes signedness: 'int' to 'size_t' (aka 'unsigned long')
521 unsafe-buffer-usage: 53032                     : []: unsafe buffer access
522 In total 83653 issues.
```

COLLECT REPORTS FROM CI BUILDS

```
4646  Sanitizer issues:
4647  SUMMARY: ThreadSanitizer: data race (/opt/gcc-14.2/lib64/libtsan.so.2+0x58bcc) in pthread_mutex_destroy
4648  SUMMARY: ThreadSanitizer: data race (/opt/gcc-14.2/lib64/libtsan.so.2+0x59f20) in free
4649  SUMMARY: ThreadSanitizer: data race (/opt/gcc-14.2/lib64/libtsan.so.2+0x5b8ce) in pthread_mutex_lock
4650  SUMMARY: ThreadSanitizer: data race (/opt/gcc-14.2/lib64/libtsan.so.2+0x5bf29) in pthread_cond_destroy
4651  SUMMARY: ThreadSanitizer: data race (/opt/gcc-14.2/lib64/libtsan.so.2+0x5da5c) in strlen
4652  SUMMARY: ThreadSanitizer: data race (/opt/gcc-14.2/lib64/libtsan.so.2+0x7873d) in recv
4653  SUMMARY: ThreadSanitizer: data race (/opt/gcc-14.2/lib64/libtsan.so.2+0x886c8) in memcpy
4654  SUMMARY: ThreadSanitizer: data race (/opt/gcc-14.2/lib64/libtsan.so.2+0xa1b4b) in operator delete[](void*)
4655  SUMMARY: ThreadSanitizer: data race (/opt/gcc-14.2/lib64/libtsan.so.2+0xa1deb) in operator delete(void*, unsigned long)
4656  SUMMARY: ThreadSanitizer: data race /opt/gcc-14.2/include/c++/14.2.0/bits/stl_vector.h:114 in std::_Vector_base<void const*, std::allocator<void const*>
      >::_Vector_impl_data::_M_copy_data(std::_Vector_base<void const*, std::allocator<void const*> >::_Vector_impl_data const&)
4657  SUMMARY: ThreadSanitizer: data race /opt/gcc-14.2/include/c++/14.2.0/bits/stl_vector.h:115 in std::_Vector_base<void const*, std::allocator<void const*>
      >::_Vector_impl_data::_M_copy_data(std::_Vector_base<void const*, std::allocator<void const*> >::_Vector_impl_data const&)
4658  SUMMARY: ThreadSanitizer: signal handler spoils errno (/opt/qt5/lib/libQt5Core.so.5+0x217580)
4659  Sanitizer summary:
4660  ASAN: 0 issues (0 unique)
4661  UBSAN: 0 issues (0 unique)
4662  TSAN: 49 issues (12 unique)
4663  ===
```

TSAN FALSE POSITIVES

```
TSAN_OPTIONS="suppressions=${CI_BUILD_DIR}/gitlab/tsan_suppressions"
```

```
mutex:libueye_api.so
deadlock:libueye_api.so
signal_handler_errno:libQt5Core.so
```

CLANG-FORMAT & EDITORCONFIG

We have a repository specific clang-format file that was modified to approximate the existing style.

We check the style in the build for every commit and set editors to format-on-save.
We **fail the build** in a pre-build verification step for violations of:

- whitespace style
- spelling errors (not yet)

Editorconfig can help to make tab/space settings consistent

- text encoding
- tab/space setting
- end of line setting
- end of file newline

https://editorconfig.org/

https://clang.llvm.org/docs/ClangFormat.html

```
# EditorConfig is awesome: https://editorconfig.org

# top-most EditorConfig file
root = true

# Unix-style newlines with a newline ending every file
[*]
end_of_line = lf
insert_final_newline = true

# Matches multiple files with brace expansion notation
# Set default charset
[*.{js,py}]
charset = utf-8

# 4 space indentation
[*.py]
indent_style = space
indent_size = 4

# Tab indentation (no size specified)
[Makefile]
indent_style = tab
```

OPEN CANARY

A set of scripts for aggregation, analysis and reporting of build and static analysis results.

Its purpose is to give you a failing CI build if new issues are introduced and to generate a team specific prioritized list of issues to work on.

https://github.com/janwilmans/OpenCanary

NOW ON TO ACTUAL CODING GUIDELINES



NO RAW LOOPS

A whole category of out-of-bounds issue is solved by using range-based for and/or ranges.

Their interface makes them very hard to use incorrectly and they allow you to express your intent more clearly which improves readability.

AVOID GLOBAL MUTABLE STATE

An incredible source of errors and hard to understand code.
Mainly due to lack of local reasoning.

To understand when state changes, you not only have to read **all functions** that access it but also know **when** they are invoked and **how.** (on different threads? how often?)

- I.2: Avoid non-`const` global variables
- I.3: Avoid singletons

- I.22: Avoid complex initialization of global objects

LIMIT SCOPE

Keep the scope of variables and type declarations as limited as possible

```
void example(bool condition)
{
    int count = 0;

    // more code

    if (condition)
    {
        count = 42;
    }
    return count;
}
```

```
void example(bool condition)
{
    // more code

    int count = 0;  // count moved here, just before it is needed
    if (condition)
    {
        count = 42;
    }
    return count;
}
```

INITIALIZE ALL VARIABLES AT DECLARATION

- C.41: A constructor should create a fully initialized object

Trivial cases:

```
int size = 0;
const int offset = 0; // const forces initialization

struct Point
{
  int x = 0;
  int y = 0;
};

Point p; // initialized by default!
```

notice that it is **no longer possible** to create an uninitialized **Point** here!

INITIALIZE ALL VARIABLES AT DECLARATION

Non-Trivial case:

```
auto result = Result::Good; // mutable
if (var > value)
{
    result = Result::Bad;
}

// instead, could be const by using the ternary operator

const auto result = (var > value) ? Result::Good : Result::Bad;
```

Notice `**result**` can be const here

INITIALIZE ALL VARIABLES AT DECLARATION

using an Immediately Invoked Function Expression (IFFE)

```
const auto details = [&client]() -> DeviceDetails {
    try
    {
        return client.GetDeviceDetails(); // 3rd party API
    }
    catch (const std::exception &)
    {
        // details not available for this client
        return {};
    }
}();
```

Notice that **`details`** can still be **const** here.
And we could also have created a free function...

USE CONST WHENEVER YOU CAN

- P.10: Prefer immutable data to mutable data

Using const on variables makes code easier to read and less error-prone because you can't assign it by mistake.

"Whenever you can" – When not?

```
1   struct terrible
2   {
3       terrible(int & value) : m_value(value)
4       {
5       }
6       int m_value;
7   };
8
9   int main()
10  {
11      int value = 0;
12      terrible a(value);
13
14      int answer = 42;
15      terrible b(answer);
16
17      b = a;
18  }
```

```
1   struct terrible
2   {
3       terrible(int & value) : m_value(value)
4       {
5       }
6       const int m_value;
7   };
8
9   int main()
10  {
11      int value = 0;
12      terrible a(value);
13
14      int answer = 42;
15      terrible b(answer);
16
17      b = a; // error!
18  }
```



https://godbolt.org/z/5dvMEvdoe

```
<source>:1:8: note: 'terrible& terrible::operator=(const terrible&)' is implicitly deleted because the default definition would be ill-formed:
    1 | struct terrible
      |        ^~~~~~~~
```

USE [[NODISCARD]]

Use [[nodiscard]] on all **pure** free functions and **const** member functions that do have side-effects

```
1   #include <string>
2
3   struct example
4   {
5       [[nodiscard]] std::string get_name() const
6       {
7           return m_name;
8       }
9       std::string m_name;
10  };
```

Rationale: const functions can't modify the object, so the only reason to call them is to get the return value.

AVOID RETURNING VALUES USING ARGUMENTS

- F.20: For "out" output values, prefer return values to output parameters
- F.21: To return multiple "out" values, prefer returning a struct

```cpp
#include <span>

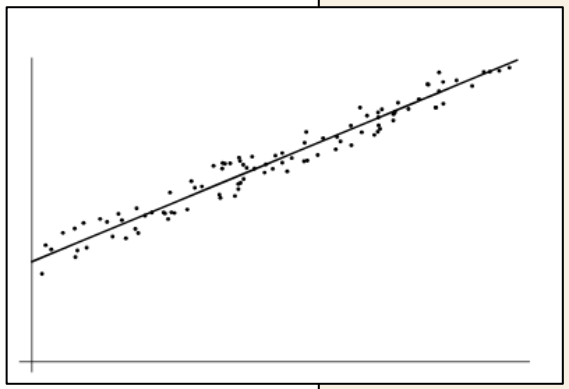
void GetLeastSquareGit(std::span<const double> edge, double & slope, double & intercept);

// instead:

struct FitResult
{
    double slope = 0.0;
    double intercept = 0.0;
};

FitResult GetLeastSquareFit(std::span<const double> edge);

const auto [slope, intercept] = GetLeastSquareFit({});
```



Rationale: breaks local reasoning for the **caller**, making reading and understanding the code harder.

https://godbolt.org/z/8r3vGn7a4

USE AUTOMATIC RESOURCE MANAGEMENT (RAII)

+ Follow the rule of 0 or the rule of 5 in that order.

- C.20: If you can avoid defining any default operations, do

```
class rule_of_zero
{
    std::string cppstring;
public:
    rule_of_zero(const std::string& arg) : cppstring(arg) {}
};
```

USE AUTOMATIC RESOURCE MANAGEMENT (RAII)

+ Follow the rule of 0 or the rule of 5 in that order.

- P.8: Don't leak any resources
- C.31: All resources acquired by a class must be released by the class's destructor

If a class requires a user-defined destructor, a user-defined copy constructor,
or a user-defined copy assignment operator,
then **implement all of them** and **add** a move constructor and move assignment operator.

```
class example
{
public:
    example(const example&) = default;
    example(example&&) = default;
    example& operator=(const example&) = default;
    example& operator=(example&&) = default;
    ~example(); // implemented
};
```

Notice: copy/move assignment operations will not be default in practice

SIMPLIFICATION: TWO KINDS OF CLASSES

Classes that have custom **destructors** deal exclusively with **ownership**, other classes should **not** need custom destructors.

```cpp
#include <thread>

struct example
{
    example(int) {}
    std::jthread m_thread;
};

int main()
{
    example a(1);
    example b(2);
    b = std::move(a);
}
```

- Single Responsibility Principle
- RAII
- Automatic Resource Management

https://en.cppreference.com/w/cpp/language/rule_of_three.html

DO NOT USE C-STYLE CASTS

In general, casts other then **static_cast** should be considered a code smell, and probably warrants the code to be separated into a function that explains a ‘why’.

Rationale: C-style casts are less explicit and more prone to errors compared to C++-style casts (static_cast, dynamic_cast, const_cast, reinterpret_cast).

C++ casts provide better type safety and are easier to search for and review. They make the intention of the cast clearer, which improves code readability and maintainability.

MORE DEFAULTS

- Avoid manual memory management using new, delete, malloc, free,...
- Do not add member variables to classes that are used as interfaces
- Do not use protected member variables.
- Avoid the use of volatile, const_cast, reinterpret_cast, typedef, register, extern, protected and va_arg
- Make all destructors of classes used in runtime polymorphism virtual

Refer to the guidelines document for details and rationales:
https://github.com/janwilmans/guidelines/blob/main/guidelines_details.md

DOCUMENT EXCEPTIONAL CODE

I.30: Encapsulate rule violations

To keep code simple and safe.

Sometimes, ugly, unsafe, or error-prone techniques are necessary
for logical or performance reasons. If so, keep them **local** and **well documented**.

LEAST SURPRISING BEHAVIOR

In summary: use **default guidelines** if you can.

Why? Because it is the **default** and therefore gives the **least surprising behavior.**

WEBSITE, LIVING DOCUMENT

https://github.com/janwilmans/guidelines

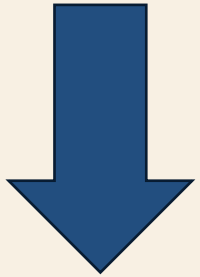
Highest Level Summary

Generally, use the most fitting tool for the job, if it has less features, it allows for less mistakes.

- [H.1] Turn on warnings! and use sanitizers [P.12] (see details here)
- [H.2] Avoid global mutable state [I.2] [I.3]
- [H.3] Keep the scope of variables and type declarations as limited as possible. examples
- [H.4] Initialize all variables at declaration. [meme]
- [H.5] Use `const` whenever you can [P.10] (but no const for member variables and return types [C.12]). [meme]
- [H.6] Use `[[nodiscard]]` for all `const` member functions returning a value.
- [H.7] Avoid returning values using arguments. [F.20] [F.21]
- [H.8] Use automatic resource management (RAII). [P.8] [C.31]
- [H.9] Follow the rule of 0 or the rule of 5 in that order.
- [H.10] Avoid owning raw pointers. [I.11] [F.26] [doc]
- [H.11] Avoid manual memory management using `new`, `delete`, `malloc`, `free`, etc.
  - When working with Qt the use of the new keyword explicitly allowed.
- [H.12] Do not use C-style casts. [meme]
- [H.13] Do not add member variables to classes used as interfaces. (Interfaces are defined as pure virtual classes that have a virtual = default destructor)
- [H.14] Do not use protected member variables.
- [H.15] Avoid the use of `volatile`, `const_cast`, `reinterpret_cast`, `typedef`, `register`, `extern`, `protected` or `va_arg`
- [H.16] Make all destructors of classes used in runtime polymorphism virtual. [C.35]
- [H.17] Avoid references as data members of a class

TAKE-AWAYS

- **defaults** are better than just guidelines
- **shared** defaults **(conventions)** are better still



Maybe in 20 years?

CONVENTIONAL C++ ?

The Default Guidelines can be found here:
https://github.com/janwilmans/guidelines

THANK YOU!

QUESTIONS?



- blog: **nullptr.nl**
- janwilmans.bsky.social
- janwilmans@toot.community